Perfecting a Final Chocobo: Exact Probabilities and Optimal Feeding Policies for *Final Fantasy XIV* Chocobo Racing

Kevin Rutledge

June 2026

Abstract

In *Final Fantasy XIV*, a fully bred "final" race chocobo caps all five attributes at 500 points. Its lifetime growth gives 245 random increments and 50 player-directed feedings, and that total cannot fill all five attributes. Some attribute must be sacrificed, conventionally *Acceleration*. I compute the probability of perfecting the other four under two regimes of play. In the **fixed-end** regime the player feeds only at rank 50. I prove that the popular 6.17% community figure is the answer to a frictionless idealization, compute the *exact* probability once feeds are treated as discrete 5/10/15-point chunks (2.30% for a pre-committed dump and 11.32% when the dump is chosen after the growth is seen), and show that perfecting *some* three attributes is certain. In the **adaptive** regime the player feeds *during* the climb and exploits the rule that a maxed attribute leaves the random pool. I cast this as a Markov decision process and solve it by exact backward induction. The best *online* probability is what a player can achieve deciding rank by rank, without foreknowledge of the rolls. For the strict Grade-3-only lineup 500/500/500/500/250 it falls between 1.06% and 1.17%, about 20% above the best fixed-deadline rule. I bracket the *offline* ceiling, what a player who could see every roll in advance could reach, exactly within [0.10%, 6.17%], and pin it at 4.48% by high-precision simulation. The closed-form and backward-induction results are exact. Three adaptive-regime quantities are Monte-Carlo estimates, flagged where they appear. The paper is self-contained, and every probabilistic tool is defined where it is first used.

Contents

1 Introduction

Chocobo racing is a long-running minigame in *Final Fantasy XIV*. A player raises a bird through fifty ranks, and its performance on the track is governed by five numerical attributes, namely *Maximum Speed*, *Acceleration*, *Endurance*, *Stamina*, and *Cunning*. Through breeding each attribute acquires a *cap*, and a fully bred "final" chocobo, the endpoint of the breeding game, caps all five at the maximum of 500 points. This paper asks one quantitative question about such a bird. *How likely is it to finish with four of its attributes exactly at the cap?*

The question is interesting precisely because the answer is far below one. An attribute grows in two ways. It receives a little *random* growth at every rank-up, and it receives *feeding*, in which the player spends a limited supply of food to add points to a chosen attribute. As the next sections make precise, the lifetime totals of 245 random increments and 50 feedings are together not enough to lift all five attributes to 500. The player must let at least one attribute fall short, and the natural sacrifice is *Acceleration*, the attribute that affects race results least and so serves as the conventional "dump." The aspirational lineup is therefore 500/500/500/500/250.

How likely the four-cap finish is depends sharply on *how* the player feeds, and I study two regimes. In the **fixed-end** regime the player banks every feeding and spends it all at rank 50. Here I explain the 6.17% figure quoted in community guides, show that it answers only a frictionless

idealization, and compute the true probability once feeds are recognized as discrete 5-, 10-, and 15-point portions. I also settle the easier goal of perfecting merely three attributes. In the **adaptive** regime the player feeds *during* the climb and exploits a subtle rule to steer the outcome. A maxed attribute stops receiving random growth. There the question becomes one of optimal control, and the quantities of interest are the best achievable success rate and the ceiling that a player able to foresee the rolls could reach.

The paper has a shared setup followed by two parts. The setup fixes the game model, the elementary probabilistic tools, and the model of a single climb. **Part I** treats fixed-end feeding. It covers the idealized 6.17% bound, the exact chunked probabilities (2.30% for a pre-committed dump and 11.32% for a dump chosen after the fact), and the certainty of a three-attribute finish. **Part II** treats adaptive feeding. It builds a Markov decision process whose exact solution brackets the online optimum of the strict Grade-3-only lineup, gives an explicit policy that realizes it, and proves exact two-sided bounds on the offline ceiling. A closing section collects every result in a single table and translates the optimal policy into a rule a player can follow. Results obtained in closed form or by exact backward induction are labeled *exact*. The few obtained by simulation are labeled *Monte-Carlo* wherever they appear.

2 The game model

The in-game rules translate into arithmetic. The rules below are as documented in the community wikis [1, 2, 3, 4, 5]. They are a fixed model with no modeling approximations beyond those noted.

Attributes and caps. A race chocobo has five attributes. Breeding raises their *caps*, and for the final chocobo studied here every cap equals the maximum, 500 points. The five caps are fixed at 500 throughout.

The percent-to-point conversion. The game expresses all growth as a percentage of an attribute's cap. At a cap of 500,

$$1\% \text{ of cap } = 5 \text{ points.}$$

Because 500 is a multiple of 100, every percentage quoted below is a whole number of points, and there is no rounding to track. (At a different cap, 1% need not be an integer, and rounding would matter. The integer-point arithmetic of this paper is special to the cap-500 final chocobo.)

Starting values. A freshly hatched rank-1 chocobo begins with every attribute at 11% of its cap, that is, at 55 points.

Random growth from ranking up. Reaching rank 50 from rank 1 takes 49 rank-ups. *Each* rank-up applies five separate +1% (+5-point) *increments*. Each increment is placed on one attribute chosen uniformly at random and independently of the others. Over a full climb the number of increments is therefore

$$N = 49 \times 5 = 245.$$

Feeding. The chocobo holds one feeding *slot* at rank 1 and gains one more at each rank-up, for 50 slots over its lifetime. Unused slots are stockpiled, so the player chooses freely *when* to spend them. Each slot holds a single feeding of grade 1, 2, or 3, which adds the points in Table 1 to one chosen attribute. A feeding may not raise an attribute above its 500 cap, and any overshoot is wasted, so to *max* an attribute the player must land on 500 exactly.

Feed grade	Growth (% of cap)	Points added (cap 500)	Slots used
Grade 1	1%	+5	1
Grade 2	2%	+10	1
Grade 3	3%	+15	1

Table 1: The three feed grades. Every feeding occupies exactly one of the 50 lifetime slots, regardless of grade, so a higher grade delivers more points per slot. Grade-3 feeding is the most slot-efficient and is central to Part II.

3 Notation and elementary tools

The mathematics is kept elementary, and the following objects are all that the arguments require. Two heavier tools are introduced in full where they are first needed, namely *generating functions* (used in Part I) and *Markov decision processes* solved by *backward induction* (used in Part II).

Factorial $n!$
the product $1 \cdot 2 \cdots n$, with $0! = 1$. It counts the number of orderings of n distinct items.

Binomial coefficient $\binom{n}{k}$
the number of ways to choose k items from n, equal to $\dfrac{n!}{k!\,(n-k)!}$.

Ceiling $\lceil x \rceil$
the smallest whole number that is not below x. For example $\lceil 3.0 \rceil = 3$ and $\lceil 3.2 \rceil = 4$. It counts "how many feedings are needed," since a partial feeding still consumes a whole slot.

Modular residue $n \bmod 3$
the remainder of n on division by 3, one of $0, 1, 2$. It tracks whether an attribute's gap to 500 is divisible by 15 and so fillable by Grade-3 feeds alone, the pivotal condition of Part II.

Probability $\mathrm{P}(E)$
a number in $[0, 1]$ measuring how often event E occurs. $\mathrm{E}[X]$ denotes the average value of a quantity X.

Binomial distribution $\mathrm{Binomial}(m, p)$
the random count of "successes" in m independent yes/no trials, each succeeding with probability p. The chance of exactly k successes is

$$\mathrm{P}(\text{count} = k) = \binom{m}{k} p^k (1-p)^{\,m-k},$$

and the average count is $m\,p$.

Multinomial distribution $\mathrm{Multinomial}(m;\, p_1, \ldots, p_r)$
the generalization to r categories. Dropping m balls independently into r boxes, box j chosen with probability p_j, gives box counts $(n_1, \ldots, n_r)$ with

$$\mathrm{P}(n_1, \ldots, n_r) = \frac{m!}{n_1!\, n_2! \cdots n_r!}\, p_1^{n_1} \cdots p_r^{n_r}.$$

Each individual box count is itself $\mathrm{Binomial}(m, p_j)$.

Poisson distribution Poisson(λ)

the count of rare independent events with average λ. The chance of exactly n is $e^{-\lambda}\lambda^n/n!$. One convenient fact about it, called *Poissonization*, is used in Part I and explained there.

4 The probabilistic model of a climb

Label the five attributes $0, 1, 2, 3, 4$ and let attribute 0 be Acceleration, the dump attribute. Write n_i for the number of random increments that land on attribute i during the climb.

Lemma 1 (Increment counts)**.** *If no attribute is maxed before rank* 50*, then the counts* $(n_0, \dots, n_4)$ *follow* Multinomial$\left(245;\, \frac{1}{5}, \dots, \frac{1}{5}\right)$*. In particular each* n_i *is* Binomial$(245, \frac{1}{5})$ *with average* 49*.*

Proof. Each of the $N = 245$ increments is placed, independently, on a uniformly chosen attribute, exactly the "balls into boxes" description of the multinomial law, with 245 balls and five equally likely boxes. The lone caveat is that the game removes an attribute from the pool once it is maxed. From random growth alone, an attribute maxes only on reaching $55 + 5n_i = 500$, that is $n_i = 89$ increments. The chance that any attribute does so is at most 1.2×10^{-8} (the bound is computed in Part I), so under the stated hypothesis the five boxes stay equally likely throughout. □

Remark 1. The hypothesis "no attribute maxed early" holds automatically in the fixed-end regime of Part I, where nothing is fed until rank 50. Part II deliberately maxes attributes *during* the climb ("locking"), which triggers the exclusion the Lemma sets aside. Its analysis accounts for that effect directly.

Deficits. Before feeding, attribute i sits at $55 + 5n_i$ points, so its gap to the cap is

$$D_i \;=\; 500 - (55 + 5n_i) \;=\; 5\,(89 - n_i) \quad \text{points.} \tag{1}$$

Feeding cost. Since D_i is a multiple of 5, the fewest slots that close the gap *exactly* use as many 15-point (Grade-3) feeds as possible and one smaller feed for any remainder. The minimum number of slots to max attribute i is therefore

$$c(n_i) \;=\; \left\lceil \tfrac{D_i}{15} \right\rceil \;=\; \left\lceil \tfrac{89 - n_i}{3} \right\rceil \qquad \text{(and 0 once } n_i \geq 89\text{).} \tag{2}$$

The ceiling is the "chunkiness" that the idealized 6.17% estimate of Part I overlooks, since a leftover of 5 or 10 points still costs a whole slot.

Success condition. With all 50 slots in hand at rank 50, maxing a chosen set T of attributes to exactly 500 succeeds precisely when their costs fit the budget,

$$\sum_{i \in T} c(n_i) \;\leq\; 50. \tag{3}$$

(Spending every slot at rank 50 makes slot *timing* automatic in the fixed-end regime. Part II reinstates timing as a genuine constraint.)

Goal	Maxed	Dump	Feed grades	Regime
Four, fixed dump	4	Acceleration (pre-set)	any	fixed-end (Part I)
Four, flexible dump	4	worst attribute	any	fixed-end (Part I)
Three	3	two worst	any	fixed-end (Part I)
Perfect	4	Acceleration (pre-set)	Grade-3 only	adaptive (Part II)

Table 2: The four goals. The first three (fixed-end) ask only that four or three attributes reach 500 using feeds of any grade. The fourth (adaptive) is stricter. It insists on the exact lineup 500/500/500/500/250 reached with Grade-3 feeds alone, with no partial feed ever wasted, and it lets the player feed during the climb.

The goals analyzed. Table 2 names the four scenarios studied in this paper and points to where each is treated. They differ along three axes, namely how many attributes are maxed, whether the sacrificed attribute is fixed in advance or chosen after the growth is seen, and whether any feed grade may be used or only the slot-efficient Grade-3.

Part I
Fixed-end feeding

In this part the player banks every feeding and spends it all at rank 50. I first show this schedule is optimal for the goals at hand (Section 5), then read off the resulting probabilities. These are the idealized 6.17% that explains the community figure (Section 6), the exact chunked values 2.30% and 11.32% (Section 7), and the certainty of a three-attribute finish (Section 8).

5 Feeding last is optimal

Proposition 1 (Optimality of feeding last)**.** *For each fixed-end goal of Table 2, which maxes a chosen set of attributes using feeds of any grade, storing every feeding until rank* 50 *maximizes the probability of success.*

Proof. The increments are dealt by chance, and the player controls only *when* to feed. Feeding can change the increment distribution in just one way. Maxing an attribute early, by Lemma 1, drops it from the pool, so that every later increment falls on the attributes that remain. For any such goal, success depends through (2)–(3) on the targets' final deficits, and a deficit shrinks only when its attribute absorbs increments. Maxing a target early freezes it and reroutes its remaining increments onto the other attributes, which enlarges their deficits or pads a dump, while maxing a dump early is never useful. Hence no early feeding lowers the required budget, and some raise it. Feeding last performs no early maxing, so it minimizes the budget needed. It also leaves the increments exactly multinomial (Lemma 1). □

Remark 2. The proof relies on the fact that, with mixed feed grades, *any* deficit can be closed exactly at rank 50. Part II abandons that freedom. Using Grade-3 feeds only, a target is maxable solely on a divisible-by-15 window, and maxing one early can *create* a window that rank-50 play would miss. That benefit is absent here, and it is exactly why adaptive feeding overtakes feed-last for the Grade-3-only goal. The optimality above is therefore specific to the any-grade goals of Part I and does not hold in general.

6 The idealized bound: where 6.17% comes from

Suppose for a moment that feedings were infinitely divisible, able to deliver any number of points rather than only the 5, 10, or 15 of a real feed. Closing a gap of D_i points would then cost $D_i/15$ slots with no ceiling. Take the "four, fixed dump" goal, $T = \{1, 2, 3, 4\}$, and write $a = n_0$ for Acceleration's count, so the four targets share the remaining $245 - a$ increments. Using the deficit (1) and the identity $\sum_{i=1}^{4} n_i = 245 - a$, the idealized cost of the four targets is

$$\sum_{i=1}^{4} \frac{D_i}{15} = \frac{5(4 \cdot 89 - (245 - a))}{15} = \frac{555 + 5a}{15} = 37 + \frac{a}{3}.$$

Requiring this to be at most 50 gives the clean condition

$$37 + \frac{a}{3} \leq 50 \iff a \leq 39. \tag{4}$$

So the idealized problem succeeds exactly when Acceleration draws at most 39 of the 245 increments. By Lemma 1, $a \sim \text{Binomial}(245, \frac{1}{5})$, so this probability is the exact finite sum

$$\mathrm{P}(a \leq 39) = \sum_{k=0}^{39} \binom{245}{k} \left(\tfrac{1}{5}\right)^{k} \left(\tfrac{4}{5}\right)^{245-k}. \tag{5}$$

Proposition 2. *The sum* (5) *equals* 6.166840% *(to six decimal places).*

Expression (5) is a closed form, a finite sum of whole numbers over 5^{245}, evaluated in exact integer arithmetic. Its value, 6.166840%, is the figure community guides round to "about 6.17%." It is the right answer to the *idealized* problem. Note that the average of a is 49, comfortably above the threshold 39, so success is a genuine "lucky low roll," an event well into the lower tail.

Remark 3. The same sum supplies the bound promised in Lemma 1. One particular attribute reaches 89 increments with probability $\sum_{k=89}^{245} \binom{245}{k}(\frac{1}{5})^{k}(\frac{4}{5})^{245-k} \approx 2.3 \times 10^{-9}$, and at most five attributes can, giving the 1.2×10^{-8} union bound quoted there.

7 The exact analysis with chunked feeding

The idealized bound overcounts successes because real feeds cannot deliver a fractional slot's worth of points. The honest success condition keeps the ceiling, that is (3) with c from (2). I now evaluate it exactly.

7.1 A counting (generating-function) approach

The exact value sums the multinomial probability of *every* increment outcome $(n_0, \ldots, n_4)$ that satisfies the success condition. These are far too many to list, so a standard bookkeeping device handles them, the *generating function*, a polynomial whose exponents store the tracked quantities. With two placeholder symbols x and y, the exponent of x records how many increments an attribute used and the exponent of y records how many slots it will cost. For one attribute,

$$g(x, y) \;=\; \sum_{n \geq 0} w(n)\, x^{\,n}\, y^{\,c(n)}, \tag{6}$$

where $w(n)$ is the probabilistic weight of receiving n increments and $c(n)$ is the feeding cost (2). Multiplying generating functions *adds* their exponents, so multiplying four copies of g enumerates

every way the four targets can split their increments while summing the costs. Reading off the coefficient of x^{245} pins the total increment count to its true value, and keeping only terms of y-exponent at most 50 enforces the budget (3). The bookkeeping is exact. It merely organizes a vast sum so a computer can evaluate it term by term.

7.2 Making the weights independent: Poissonization

One obstacle remains. In the multinomial law the five counts must add to 245, so the attributes are dependent and their generating functions do not cleanly multiply. A classical trick, *Poissonization*, removes the link, resting on one fact.

> If five independent Poisson(49) counts are observed and only the outcomes that happen to sum to 245 are kept, the resulting counts follow exactly Multinomial(245; $\frac{1}{5}, \ldots, \frac{1}{5}$).

Independent Poisson counts "do not know" about one another, so they multiply freely, and conditioning on their sum afterwards rebuilds the desired multinomial. Concretely, for every event S on the counts,

$$\mathrm{P}(S) \;=\; \frac{\mathrm{P}\left(S \text{ and } \sum_i N_i = 245\right)}{\mathrm{P}\left(\sum_i N_i = 245\right)}, \qquad N_i \overset{\text{ind.}}{\sim} \text{Poisson}(49). \tag{7}$$

The denominator is a single value. A sum of independent Poisson(49) counts is Poisson(245), so $\mathrm{P}(\sum_i N_i = 245) = e^{-245}245^{245}/245!$. In the numerator the attributes are independent, so $w(n) = e^{-49}49^n/n!$ is used in (6), and the generating-function product applies directly.

7.3 Evaluation and results

For the four-fixed-dump goal the computation forms $g(x, y)^4$ (the four targets), multiplies by $h(x) = \sum_n w(n)x^n$ for the unconstrained Acceleration attribute, extracts the coefficient of x^{245}, sums the terms of y-exponent at most 50, and divides by the denominator of (7). This finite computation was carried out exactly, organized as a table of running coefficients. Two internal checks confirm it. The total probability over *all* outcomes returns 1.0000000000, and replacing the ceiling (2) by the idealized cost reproduces Proposition 2's 6.166840% to the digit.

Theorem 1 (Four, fixed dump)**.** *Under feed-last play, with Acceleration pre-committed as the dump, the probability of maxing the other four attributes to exactly* 500 *is*

$$\boxed{2.301513\%}.$$

The *flexible-dump* goal maxes any four attributes, discarding whichever proves most expensive. It uses the same machinery with the bookkeeping extended to also track the single largest per-attribute cost, so the worst attribute can be dropped after the growth is seen. Its probability is 11.324144%. The gap between 2.30% and 11.32% is the price of committing to Acceleration in advance rather than sacrificing whatever rolls worst.

8 Maximizing three attributes is certain

Theorem 2 (Three attributes)**.** *Under feed-last play, the three highest-count attributes can always be maxed, and the three-attribute goal succeeds with probability exactly* 1*.*

Proof. Sort the counts $n_{(1)} \geq n_{(2)} \geq n_{(3)} \geq n_{(4)} \geq n_{(5)}$. The two smallest are each at most the average 49, so $n_{(4)} + n_{(5)} \leq \frac{2}{5} \cdot 245 = 98$, and therefore

$$n_{(1)} + n_{(2)} + n_{(3)} \;\geq\; 245 - 98 \;=\; 147.$$

By the deficit formula (1), the combined deficit of the three largest attributes is at most

$$\sum_{\text{top 3}} D_i = 5\big(3 \cdot 89 - (n_{(1)} + n_{(2)} + n_{(3)})\big) \leq 5\,(267 - 147) = 600 \text{ points.}$$

Each ceiling obeys $\lceil D_i/15 \rceil \leq D_i/15 + \frac{14}{15}$, so the total feeding cost is bounded by

$$\sum_{\text{top 3}} \left\lceil \tfrac{D_i}{15} \right\rceil \;\leq\; \frac{600}{15} + 3 \cdot \frac{14}{15} \;=\; 40 + 2.8 \;=\; 42.8,$$

hence at most 42 slots, an integer. Since $42 \leq 50$, the budget always suffices, whatever the random outcome. □

This is why three-attribute builds are treated as routine in practice. They are guaranteed, well beyond merely likely. The harder goal of maxing three *specific* pre-chosen attributes, where the two dump attributes may hoard increments, is only near-certain rather than certain, and is not analyzed here.

Part II
Adaptive feeding

In this part the player feeds *during* the climb rather than only at rank 50. That freedom unlocks a stricter target and turns the analysis into one of optimal control. This part describes the opportunity (Section 9), casts the climb as a Markov decision process and solves it for the best *online* policy (Section 10), states the explicit policy a player can follow (Section 11), bounds the *offline* ceiling exactly (Section 12), and examines the stricter exact lineup 500/500/500/500/250 (Section 13).

9 The adaptive opportunity

A stricter goal. Call a finish *perfect* if the four targets reach exactly 500 using *only* Grade-3 feeds, with no 5- or 10-point top-off ever spent, landing the lineup 500/500/500/500/250. By the cost formula (2), a target is maxable with Grade-3 feeds alone only when its count satisfies $n_i \equiv 2 \pmod 3$ at the moment it is fed (then its deficit $D_i = 5(89 - n_i)$ is a multiple of 15). This is strictly harder than the any-grade goal of Part I, and the surcharge for "no wasted feed" is exactly this divisibility.

A smarter move. As an attribute absorbs increments its count cycles through residues $0 \to 1 \to 2 \to 0 \to \cdots$, so a *window* ($n_i \equiv 2$) reopens every three increments. Rather than gamble that a target lands on a window at rank 50, the player can *lock* it mid-climb and feed it to 500 at a window. Locking carries a cost. By Lemma 1 a maxed attribute leaves the random pool, so every later rank-up's five increments fall on the attributes that remain, Acceleration included. Locking early therefore trades "catch a good window now" against "inflate the dump later," and deciding when to lock each target is a sequential decision problem.

Online versus offline. Two yardsticks bracket any policy. An *offline* player who could see every future roll would lock each target at the last window that keeps Acceleration in budget, and this ceiling is $\approx 4.48\%$, bounded exactly in Section 12. A real *online* player decides from the current state alone and does worse. The distance between them is the *prophet gap*, the value of information, which no online rule can recover [7].

10 The decision process and a budget identity

The climb is modeled as a finite-horizon *Markov decision process*. At each rank-up the five increments land (Lemma 1), then the player may lock any target sitting on a window. The best *online* policy is found by *backward induction*. Working from the final rank-up backward, the best action in each state solves $V_t(s) = \max_a\{\, r_t(s,a) + \mathrm{E}[V_{t+1}] \,\}$ [6].

The state looks intractable. Four target counts (each 0–88), Acceleration's count, the slot balance, and the rank give on the order of 10^{11} combinations. One identity collapses it. A target locked at count c_i uses $(89 - c_i)/3$ feeds, and since every increment lands on Acceleration or on a target before it locks, $\sum_i c_i + a = 245$. Hence the total feeds spent is

$$\sum_{i=1}^{4} \frac{89 - c_i}{3} = \frac{356 - \sum_i c_i}{3} = \frac{111 + a}{3}, \tag{8}$$

independent of when the locks happen. So the 50-slot budget is met exactly when $a \le 39$, the very condition the Acceleration cap already imposes. The feed count therefore never needs tracking, and each target collapses to its residue $n_i \bmod 3$ (only "on a window?" matters). Dropping the secondary requirement that slots be physically accrued by lock time (verified below to bind on just 0.17% of runs), the state shrinks to

$$(\text{rank},\ a,\ \text{multiset of unlocked residues}),$$

fewer than 10^5 values, an exact backward induction that runs in milliseconds.

10.1 The optimal value

Backward induction on this state yields the optimal value of the (slot-relaxed) online problem,

$$\boxed{1.170\%}\ \ (\text{Acceleration} \le 250), \qquad 0.979\%\ \ (\text{Acceleration} = 250).$$

Two internal checks support it. Under a fixed feed-last policy the same transition model reproduces the exact Grade-3-only value 0.10239%, and the optimum exceeds the best fixed-deadline rule of Section 11, as it must. Because the relaxation only *removes* a constraint, 1.170% is an *upper bound* on the true online optimum. Replaying the optimal residue-policy with the slot-timing constraint reinstated achieves 1.063% (a Monte-Carlo estimate), a matching lower bound. Therefore

$$1.063\% \ \le\ (\text{true online optimum}) \ \le\ 1.170\%.$$

11 The optimal policy is an Acceleration-aware threshold

The optimal action has a clean shape. *Lock a target the moment it reaches a window, provided rank* $\ge T(a)$, where the threshold T rises with the Acceleration count a (Table 3). When Acceleration

Acceleration count a	Acceleration value	lock from rank $T(a)$
0	55	39
8	95	41
16	135	42
24	175	44
30	205	45
36	235	47
39	250	49

Table 3: Optimal lock threshold with four targets still unlocked. Feed a target to 500 on its divisible-by-15 window once rank $\geq T(a)$. With fewer targets left, lock about one rank earlier.

is low there is budget to spare, so banking a window early pays off. When Acceleration is tight one must wait until the final rank or two. A simple fit is $T(a) \approx 39 + a/5$.

This is why a *fixed* deadline is suboptimal. The best fixed rule, "never lock before rank D, then lock on sight," peaks at $D = 47$ and scores 0.885% (Monte-Carlo). It is too patient in the low-Acceleration states, where the optimum locks as early as rank 40. The Acceleration-aware rule recovers that, raising success by about 20% (0.885% $\rightarrow$ 1.063%), roughly one perfect bird in 90 rather than one in 113.

12 The offline ceiling, bounded

The offline ceiling is the only figure in this paper not computed exactly, and it is worth seeing why. It is an *offline* quantity, the fraction of climbs for which *some* lock schedule wins *given foreknowledge of every roll*. Backward induction cannot reach it, because the optimal decision depends on the future and, through exclusion, each decision reroutes that very future. So it is *bracketed* exactly, then pinned by simulation.

Exact upper bound. Remove the exclusion side effect, and let a locked target keep absorbing its own increments. Then Acceleration keeps its natural count A, and every target passes through a window (at counts $2, 5, 8, \dots$) for free, so success reduces to the budget $A \leq 39$ alone. Since this can only help, by Proposition 2

$$P_{\text{clair}} \;\leq\; \mathrm{P}(A \leq 39) \;=\; 6.166840\%.$$

Exact lower bound. Feed-last is a concrete feasible schedule, so its success rate is a rigorous floor. That rate requires all four counts $\equiv 2 \pmod 3$ and $A \leq 39$, and equals 0.10239% (the value validated in Section 10). Combining,

$$0.10239\% \;\leq\; P_{\text{clair}} \;\leq\; 6.16684\%.$$

Pinning the value. Inside this bracket the value is located by high-precision Monte Carlo. For each climb an exhaustive lock-schedule search runs on its fixed future, exactly the best a player who could see every roll would do. With $N = 10^6$ runs (reproducible seed),

$$P_{\text{clair}} = 4.476\% \pm 0.04\% \qquad \text{(Wilson 95\% interval [4.44\%, 4.52\%])},$$

comfortably inside the proven bracket. The two exact bounds together with this confidence interval are the rigorous statement of the ceiling, and a closed form is not available, for the reason above. The prophet gap is the value of foreknowledge, a factor of roughly four between the online optimum ($\leq 1.170\%$) and this ceiling, and no online policy can recover it.

13 The exact lineup: Acceleration at exactly 250

The goal of Part II so far asked only that Acceleration finish at *or below* 250. A stricter reading insists on the literal lineup 500/500/500/500/250, with Acceleration at *exactly* 250 (count $a = 39$). The numbers change sharply, and one structural fact explains why.

At the exact lineup the feed rule is moot. If $a = 39$ the four targets share $245 - 39 = 206$ increments, so by the budget identity (8) their total deficit is $5(4 \cdot 89 - 206) = 750$ points. Closing 750 points in 50 slots forces an average of exactly 15 points per slot, so *every* feed must be Grade-3, and each target's deficit must itself be a multiple of 15. Mixed feed grades buy nothing here. The exact lineup is intrinsically a Grade-3-only target, and the "≤ 250" slack that mixed feeds exploit is gone.

The probabilities. Pinning Acceleration to exactly 250, the methods of Sections 10–12 give the following, alongside the 0.979% relaxed online optimum already recorded in Section 10.

- **Feed-last** reaches 0.067% (exact). It needs both $a = 39$ and all four residues aligned, with no adaptivity to help.
- **Online optimum** lies between 0.847% (achievable, real slot-timing, Monte-Carlo) and 0.979% (relaxed, exact), the same lower and upper sandwich as the ≤ 250 case, only stricter.
- **Offline** reaches 4.36% (Monte-Carlo, Wilson 95% interval [4.28%, 4.46%]).

Reading against the 2.30% benchmark. It is tempting to compare the exact lineup to the 2.301513% four-attribute figure of Part I, but they are different targets. The 2.30% figure asks for four caps with Acceleration anywhere ≤ 250 and *any* feed grade, fed at the end. Pinning Acceleration to exactly 250 sits at the budget edge and discards that slack, so a real-time player cannot approach 2.30%, and the online ceiling is about 1%. Only an offline player, who can deliberately inflate Acceleration up to exactly 250 by locking targets early, exceeds it (at $\approx 4.36\%$), and that requires foreknowledge, so it is not playable. In short, the literal 500/500/500/500/250 lineup is a vanity constraint. Since Acceleration is the dump attribute, ending at 230 is as good as 250, and the achievable goal worth pursuing remains the ≤ 250 four-attribute target of Part I.

14 Results and discussion

Table 4 collects every probability computed in the paper, with the method that produced it.

Reading the fixed-end numbers. The headline 2.301513% means that a player who commits to dumping Acceleration and feeds at the end perfects the other four attributes on about one final chocobo in $1/0.0230 \approx 43$. The shortfall from the idealized 6.17% is entirely feed chunkiness. A leftover of 5 or 10 points wastes a whole slot, so the real budget is tighter than a points count

Scenario	Play	Value	Method
Fixed-end feeding (Part I)			
Idealized bound	frictionless $a \leq 39$	6.166840%	exact (binomial sum)
Four, fixed dump	max 4, Acceleration dumped	2.301513%	exact (gen. function)
Four, flexible dump	max 4, worst dumped	11.324144%	exact (gen. function)
Three attributes	max best 3	100%	exact (closed form)
Adaptive, Grade-3-only perfect, Acceleration ≤ 250 *(Part II)*			
Feed-last	lock only at rank 50	0.10239%	exact
Best fixed deadline	lock from rank 47	0.885%	Monte-Carlo
Online optimum	best state-based policy	1.063%–1.170%	MC (lower), exact (upper)
Offline ceiling	foresee every roll	4.476%	MC; bounds exact
Adaptive, exact lineup 500/500/500/500/250 *(Part II)*			
Feed-last	lock only at rank 50	0.067%	exact
Online optimum	best state-based policy	0.847%–0.979%	MC (lower), exact (upper)
Offline	foresee every roll	4.36%	Monte-Carlo

Table 4: All results. The ≤ 250 offline ceiling is bracketed exactly within [0.10239%, 6.16684%] and pinned by simulation at 4.476% (Wilson 95% interval $[4.44\%, 4.52\%]$, $N = 10^6$). The exact-lineup offline value is the analogous Monte-Carlo estimate. All figures are exact except the adaptive-regime Monte-Carlo values noted, namely the best fixed deadline, the online achievable lower bounds, and the offline point values.

suggests. Refusing to pre-commit, and dumping whichever attribute rolls worst, raises the rate to 11.32% (about one in nine), and merely perfecting the best *three* attributes is certain.

Reading the adaptive numbers. The strict Grade-3-only lineup 500/500/500/500/250 is far rarer, but *when* the player feeds matters enormously. Feeding only at rank 50 succeeds just 0.10239% of the time. An Acceleration-aware lock-timing policy lifts the online optimum to between 1.063% and 1.170% (about one in 90), roughly 20% above the best fixed deadline. An offline player who could foresee the rolls would reach 4.476%, but the gap from the online optimum to that ceiling is the prophet gap, unrecoverable without foreknowledge.

A rule of thumb for play. For the any-grade goals of Part I, the advice is simply to bank all feeds and spend them at rank 50. One may also abandon a doomed bird early, since with k rank-ups left the four targets' combined deficit can fall by at most $40k$ points, so once it exceeds $40k$ the bird cannot recover. For the Grade-3-only lineup of Part II, watch Acceleration. If its value passes 250 the perfect finish is already impossible. Otherwise feed a target to 500 the moment its gap is a multiple of 15 and the rank has reached the threshold $T(a)$ of Table 3.

Exact versus simulation. Every fixed-end figure, the online optima's upper bounds, the feed-last values, and the ≤ 250 offline ceiling's two bounds are exact. The remaining figures resist closed form and are estimated by simulation. These are the best fixed deadline (0.885%), the online achievable lower bounds (1.063% for ≤ 250 and 0.847% for the exact lineup), and the offline point values (4.476% and 4.36%). Each is flagged in Table 4 and where it is derived.

15 Conclusion

I modeled the climb of a final *Final Fantasy XIV* race chocobo as a multinomial allocation of 245 random growth increments, refined by a 50-slot feeding budget delivering 5, 10, or 15 points per slot, and studied the chance of perfecting four attributes under two regimes of play.

In the **fixed-end** regime the player banks every feed and spends it at rank 50. I proved this schedule optimal for the any-grade goals, showed that the familiar 6.17% community figure answers only a frictionless idealization, and computed the true probabilities once feeds are recognized as discrete chunks. These are 2.301513% for a pre-committed Acceleration dump, 11.324144% when the dump is chosen after the growth is seen, and certainty for the three-attribute goal. Each of these is exact, by a binomial sum, a generating-function enumeration, and an elementary inequality.

In the **adaptive** regime the player feeds during the climb, exploiting the rule that a maxed attribute leaves the random pool. Casting the climb as a Markov decision process and collapsing its state through the identity that total feeds depend only on the final Acceleration count, exact backward induction brackets the best online probability of the strict Grade-3-only perfect (four maxed, Acceleration ≤ 250) between 1.06% and 1.17%. That optimum is realized by an Acceleration-aware lock-timing threshold, which locks a target on its divisible-by-15 window once the rank reaches $T(a)$, and it beats any fixed deadline by about 20%. An offline player who could foresee the rolls would reach 4.48%, exactly bracketed within $[0.10\%, 6.17\%]$, and the gap from the online optimum to that ceiling is the prophet gap, the value of information no real player commands. Demanding the *exact* lineup 500/500/500/500/250 is stricter still. At that budget edge the feed grade becomes irrelevant (every feed must be Grade-3), the online ceiling falls to about 1% (0.85–0.98%), and only an offline player, able to inflate Acceleration up to exactly 250, reaches $\approx 4.4\%$.

For a player, the practical reading is plain. Three capped attributes come for free, four are a one-in-43 to one-in-nine lottery depending on how the dump is chosen, and the Grade-3-only perfect (Acceleration ≤ 250) is about one bird in ninety even under optimal timing. The literal 500/500/500/500/250 lineup is rarer still and not worth the fuss, since a lower Acceleration is just as good. All figures are exact except the handful of adaptive-regime simulation estimates, flagged throughout.

A Model constants

Quantity	Value
Attribute cap (final chocobo)	500 points
Points per 1% of cap	5
Starting value per attribute (rank 1)	55 points (11%)
Rank-ups from rank 1 to 50	49
Random increments per rank-up	5 (each +5 points)
Total random increments N	245
Feeding slots over a lifetime	50
Points per feeding (grades 1/2/3)	5 / 10 / 15
Increments to max one attribute by chance	89

B Full lock-threshold table

Table 3 in the main text lists the optimal lock threshold $T(a)$ for four still-unlocked targets at a coarse grid. Table 5 gives the threshold for every Acceleration count a from 0 to 39 and for each

number u of still-unlocked targets. A target on a divisible-by-15 window should be locked once rank $\geq T$.

a	Accel. value	$u = 4$	$u = 3$	$u = 2$	$u = 1$
0	55	39	39	40	42
1	60	39	40	40	42
2	65	40	40	41	42
3	70	40	40	41	42
4	75	40	40	41	42
5	80	40	40	41	43
6	85	40	41	41	43
7	90	41	41	42	43
8	95	41	41	42	43
9	100	41	41	42	43
10	105	41	41	42	44
11	110	41	42	42	44
12	115	42	42	43	44
13	120	42	42	43	44
14	125	42	42	43	44
15	130	42	42	43	45
16	135	42	43	43	45
17	140	43	43	44	45
18	145	43	44	44	45
19	150	43	44	44	45
20	155	43	44	44	46
21	160	43	44	44	46
22	165	44	44	45	46
23	170	44	45	45	46
24	175	44	45	45	46
25	180	44	45	45	47
26	185	44	45	45	47
27	190	45	45	46	47
28	195	45	46	46	47
29	200	45	46	47	47
30	205	45	46	47	48
31	210	45	46	47	48
32	215	46	46	47	48
33	220	46	47	47	48
34	225	47	47	48	48
35	230	47	48	48	49
36	235	47	48	48	49
37	240	47	48	48	49
38	245	47	48	48	49
39	250	49	48	49	49

Table 5: Optimal lock threshold $T(a, u)$ from the backward-induction policy, for *every* Acceleration count a from 0 to 39. Lock a windowed target once rank $\geq T$. The threshold rises with Acceleration a (less budget to spare) and falls slightly as fewer targets remain (u smaller). The lone exception is the degenerate boundary $a = 39$, where Acceleration already sits at its 250-point ceiling and any further increment dooms the run, so the thresholds there are effectively moot.

C Reproducibility

The exact results use only finite arithmetic. The binomial sum (5) is evaluated in exact integers. The chunked probabilities (Section 7) and the online optimum (Section 10) are generating-function convolutions and backward inductions over fewer than 10^5 states, carried with their internal checks (total probability 1, and reproduction of the idealized 6.166840% and the feed-last 0.10239%). The

three Monte-Carlo figures use fixed seeds and the sample sizes quoted where they appear, most notably $N = 10^6$ for the offline ceiling, reported with a Wilson 95% interval. No result depends on unstated randomness.

References

[1] *Chocobo Racing*, Final Fantasy XIV Online Wiki (Console Games Wiki). https://ffxiv.consolegameswiki.com/wiki/Chocobo_Racing

[2] *Rank-Ups: Attribute Gains*, FFXIV Chocobo Racing. https://ffxivchocoboracing.wordpress.com/2015/06/01/intermediate-rank-ups-attribute-gains/

[3] *Advanced: Raising That Final Chocobo*, FFXIV Chocobo Racing. https://ffxivchocoboracing.wordpress.com/2015/06/02/advanced-raising-that-final-chocobo/

[4] *Training*, FFXIV Chocobo Racing. https://ffxivchocoboracing.wordpress.com/2015/05/26/intermediate-training/

[5] *Chocobo Racing and Breeding Guide*, Icy Veins. https://www.icy-veins.com/ffxiv/chocobo-racing-and-breeding-guide

[6] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, Wiley, 1994.

[7] T. P. Hill and R. P. Kertz, *A survey of prophet inequalities in optimal stopping theory*, Contemp. Math. **125** (1992), 191–207.